CONCEPTION ET ARCHITECTURE DU PROJET

SAE 3.02 — Mini-simulation d'un carrefour connecté

Étudiants	Jules Landauer et Mael [Nom]
Formation	BUT Réseaux & Télécommunications — Parcours ROM
Temps disponible	38,5 heures pour le binôme
Version	1.0

1. Objectif de la conception

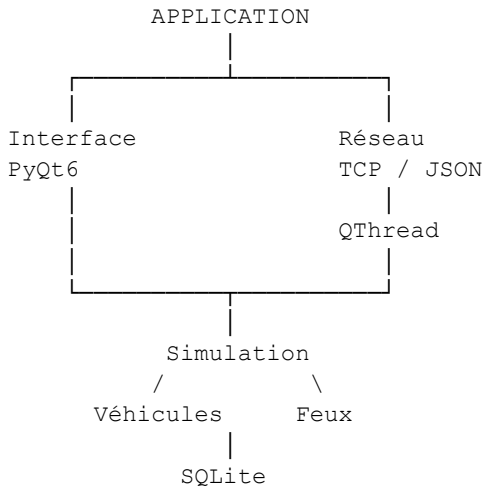
Cette étape permet de définir une architecture simple avant de commencer le développement. Le projet est volontairement limité afin de rester réalisable dans le temps disponible et avec le niveau actuel des deux étudiants.

L'objectif est de réaliser une petite application PyQt6 représentant un carrefour, quelques véhicules et des feux de circulation, avec une communication réseau TCP entre un client et un serveur. Une base SQLite permettra également d'enregistrer quelques événements.

2. Fonctionnement général

1. L'utilisateur lance l'application graphique.
2. Le carrefour et ses feux sont affichés.
3. La simulation peut être démarrée ou arrêtée.
4. Des véhicules peuvent être ajoutés à la simulation.
5. Les feux changent automatiquement d'état.
6. Les véhicules avancent et s'arrêtent lorsque le feu correspondant est rouge.
7. Le module réseau permet d'envoyer et de recevoir de petits messages JSON.
8. Les événements importants peuvent être enregistrés dans SQLite.

3. Architecture générale



L'interface graphique reste dans le thread principal. Les opérations réseau sont placées dans un QThread afin d'éviter de bloquer l'application.

4. Organisation des fichiers

```
SAE-3.02/
├── README.md
├── requirements.txt
├── src/
│   ├── main.py
│   ├── gui.py
│   ├── vehicle.py
│   ├── traffic_light.py
│   ├── simulation.py
│   ├── server.py
│   ├── client.py
│   └── database.py
├── database/
│   └── traffic.db
└── docs/
    ├── Cahier_des_Charges.pdf
    ├── Bilan_Projet.pdf
    ├── IAGraphie.pdf
    └── Releves_Decisions/
```

5. Rôle des fichiers

Fichier	Rôle	Responsable principal
main.py	Lancement de l'application et création de la fenêtre.	Jules
gui.py	Fenêtre PyQt6, boutons, affichage et signaux.	Jules
vehicle.py	Classe représentant un véhicule.	Mael
traffic_light.py	Classe et états des feux.	Mael
simulation.py	Gestion de l'évolution de la simulation.	Jules + Mael
server.py	Serveur TCP recevant les messages.	Jules
client.py	Client TCP envoyant les messages.	Mael
database.py	Création et utilisation de SQLite.	Mael

6. Interface graphique prévue

- zone centrale : représentation du carrefour ;
- zone de contrôle : boutons Démarrer, Arrêter et Ajouter une voiture ;
- zone d'informations : nombre de véhicules et état des feux ;

- zone réseau : état de la connexion et dernier message reçu.

L'interface restera volontairement simple. Une maquette précise pourra être réalisée avant le développement.

7. Classes principales

7.1 Vehicle

```
class Vehicle:
    def __init__(self, vehicle_id, x, y, direction):
        ...
```

La classe Vehicle contient les informations minimales nécessaires à l'affichage et au déplacement d'un véhicule.

7.2 TrafficLight

```
class TrafficLight:
    def __init__(self, name):
        self.name = name
        self.state = "RED"
```

La classe TrafficLight gère le nom du feu et son état. Les états utilisés seront RED, ORANGE et GREEN.

7.3 Simulation

La classe Simulation gère la liste des véhicules et l'évolution de leur position. Elle coordonne également les informations provenant des feux.

8. Communication réseau

Pour rester à un niveau adapté au projet, une seule communication TCP simple sera mise en place.

```
Serveur
  |
  | TCP : 127.0.0.1:5000
  |
Client
```

Les messages seront encodés en JSON. Exemple :

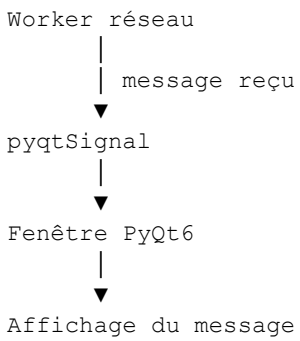
```
{"type": "vehicle", "id": 3, "speed": 30}
```

Types de messages prévus :

- vehicle : information concernant un véhicule ;
- traffic_alert : information sur un feu ;
- status : état général de la simulation.

9. Thread réseau et signaux Qt

Le serveur ou le client réseau pourra fonctionner dans un QThread. Lorsqu'un message est reçu, le worker émet un signal Qt. La fenêtre principale reçoit ensuite ce signal et met à jour l'affichage.



Cette organisation respecte la contrainte de ne pas modifier directement les widgets depuis un thread secondaire.

10. Base de données SQLite

Une seule table principale sera suffisante pour le projet :

```
events
-------------------------
id
date
type
vehicle_id
message
```

Les événements enregistrés pourront être : ajout d'un véhicule, alerte liée à un feu ou message réseau reçu.

11. Technologies utilisées

Technologie	Utilisation	Obligatoire
Python 3.x	Langage principal	Oui
PyQt6	Interface graphique	Oui
socket	Communication TCP	Oui
json	Format des messages	Oui
QThread / pyqtSignal	Réseau sans bloquer l'IHM	Oui
SQLite	Stockage des événements	Autorisé
Git / GitHub	Versionnement et rendu	Oui

12. Répartition du développement

La répartition est volontairement équilibrée et chaque étudiant participera aux tests et à l'intégration.

Jules	Mael	Travail commun
IHM PyQt6 Serveur TCP QThread réseau main.py Une partie des tests	Véhicules Feux Client TCP SQLite Une partie des tests	Architecture Simulation Intégration Tests finaux README et GitHub

13. Plan de développement — 38,5 h

Étape	Jules	Mael	Total
Analyse et architecture	1 h	1 h	2 h
Interface PyQt6	3 h	2 h	5 h
Feux et simulation	1 h	2 h	3 h
Véhicules	2 h	2 h	4 h
Communication socket	3 h	3 h	6 h
QThread et signaux	2 h	2 h	4 h
SQLite	0 h	2 h	2 h
Tests et corrections	3 h	2 h	5 h
README / documentation	1 h	1 h	2 h
Intégration finale / GitHub	3 h	2,5 h	5,5 h

TOTAL : 19 h pour Jules + 19,5 h pour Mael = 38,5 h pour le binôme.

14. Ordre de développement

9. Créer le dépôt GitHub et l'environnement virtuel.
10. Créer la fenêtre PyQt6 minimale.
11. Afficher le carrefour.
12. Créer la classe TrafficLight et faire fonctionner les changements d'état.
13. Créer la classe Vehicle et afficher quelques véhicules.
14. Ajouter le déplacement simple des véhicules.
15. Créer le serveur TCP.
16. Créer le client TCP.
17. Faire communiquer le réseau avec l'application.
18. Déplacer la partie réseau dans un QThread et utiliser un signal Qt.
19. Ajouter SQLite.
20. Tester l'ensemble.
21. Nettoyer le code, ajouter les docstrings et terminer le README.

15. Ce qui reste hors périmètre

- plusieurs carrefours simultanés
- simulation réaliste de la circulation
- algorithmes complexes d'optimisation des feux
- intelligence artificielle ou machine learning
- communication avec un serveur distant
- authentification des utilisateurs
- interface graphique très poussée
- statistiques avancées

Ces éléments ne sont pas retenus afin de respecter le temps disponible de 38,5 heures.

16. Critères de réussite

- L'application démarre correctement.
- Le carrefour est visible dans l'interface.
- Les feux changent d'état.
- Les véhicules peuvent être ajoutés et se déplacer.
- Le client et le serveur peuvent communiquer en TCP.
- Les messages sont transmis en JSON.
- La communication réseau ne bloque pas l'interface.
- Les événements principaux peuvent être enregistrés dans SQLite.
- Le projet peut être installé avec requirements.txt.
- Le README permet à une autre personne de lancer le projet.

17. Suite du projet

Une fois cette conception validée, le développement commencera par l'environnement Python et l'interface PyQt6. Les fonctionnalités seront ajoutées progressivement et testées au fur et à mesure.